

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Algoritma Sorting”

disusun oleh:

Silva Nurifvan

2511533004

Dosen pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2025

KATA PENGANTAR

Segala puji dan syukur senantiasa kita panjatkan ke hadirat Allah SWT, karena hanya berkat limpahan rahmat, karunia, serta kemudahan yang diberikan-Nya, kami akhirnya dapat menyelesaikan penyusunan laporan dan tugas berjudul “Algoritma Sorting” ini dengan baik dan lancar sesuai dengan waktu yang telah ditentukan. Tak lupa, semoga shalawat serta salam senantiasa tercurah kepada junjungan kita Nabi Besar Muhammad SAW, beserta keluarga, sahabat, dan seluruh pengikutnya yang setia mengajarkan ajaran beliau hingga akhir zaman.

Laporan ini kami susun guna memenuhi salah satu syarat penyelesaian mata kuliah Praktikum Struktur Data, sekaligus sebagai wujud penerapan ilmu pengetahuan yang telah kami dapatkan selama perkuliahan berlangsung. Melalui penyusunan tugas ini, kami berharap dapat memberikan gambaran yang jelas mengenai cara kerja serta penerapan berbagai algoritma pengurutan data dalam pemrograman.

Kami sangat menyadari bahwa laporan ini masih jauh dari kata sempurna, baik dari segi kedalaman materi, tata bahasa, maupun cara penyajiannya. Hal ini disebabkan oleh keterbatasan pengetahuan dan pengalaman yang kami miliki. Oleh karena itu, kami sangat mengharapkan kritik serta saran yang bersifat membangun dari semua pihak, guna memperbaiki kekurangan yang ada agar dapat menjadi bekal dan perbaikan untuk tugas-tugas kami di masa mendatang.

Sebagai penutup kata, kami mengucapkan terima kasih yang sebesar-besarnya kepada Bapak/Ibu Dosen pengampu mata kuliah ini, yang telah meluangkan waktu, tenaga, dan pikiran untuk memberikan bimbingan, arahan, serta penjelasan yang sangat bermanfaat selama proses pembelajaran maupun penyelesaian tugas ini. Semoga apa yang telah kami susun ini dapat memberikan manfaat bagi kami sebagai penyusun maupun bagi pembaca, serta dapat menambah wawasan mengenai ilmu pemrograman dan struktur data.

Padang, 21 Mei 2026

Silva Nurifvan

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data atau yang sering disebut dengan istilah *Sorting* adalah salah satu operasi yang paling umum digunakan dalam dunia pemrograman. Dalam praktikum ini, saya mempelajari tiga metode dasar pengurutan, yakni Bubble Sort, Insertion Sort, dan Selection Sort. Ketiga algoritma ini tergolong dalam kelompok *comparison-based sorting*, yang berarti proses pengurutannya dilakukan dengan cara membandingkan satu elemen data dengan elemen lainnya.

Cara kerja masing-masing algoritma pun berbeda: Bubble Sort memproses data dengan menukar posisi dua elemen yang berdekatan jika urutannya belum benar. Sementara itu, Insertion Sort bekerja dengan memasukkan setiap elemen ke tempat yang sesuai dalam susunan data. Sedangkan Selection Sort dilakukan dengan menentukan elemen yang bernilai paling kecil, lalu memindahkannya ke posisi awal. Setiap metode ini memiliki keunggulan dan keterbatasannya masing-masing, yang disesuaikan dengan keadaan data yang akan diolah.

Selain menyusun program yang berjalan di tampilan teks, pada praktikum ini saya juga mengembangkan program berbasis antarmuka grafis atau GUI. Tujuannya adalah untuk menampilkan tahapan proses Insertion Sort secara bertahap, sehingga cara kerjanya menjadi lebih mudah dipelajari dan dipahami.

1.2 Tujuan

Tujuan dilaksanakannya praktikum ini adalah agar saya memahami prinsip kerja dari ketiga algoritma pengurutan tersebut. Saya diharapkan dapat menerapkannya ke dalam kode program menggunakan bahasa pemrograman Java, baik yang berjalan di tampilan konsol maupun dalam bentuk antarmuka grafis. Selain itu, saya juga belajar menilai seberapa efektif dan efisien setiap algoritma dengan mengamati berapa kali proses pengulangan dan pertukaran data yang terjadi.

Melalui program GUI untuk Insertion Sort ini, saya juga memahami cara menampilkan proses pengurutan secara berurutan, sehingga jalannya algoritma dapat diamati dan diikuti dengan jelas.

1.3 Manfaat

Dari kegiatan praktikum ini, saya mendapatkan pemahaman yang lebih baik dan mendalam mengenai berbagai cara mengatur urutan data. Saya menyadari bahwa tidak semua algoritma pengurutan cocok digunakan untuk segala jenis kondisi data. Sebagai contoh,

Bubble Sort sangat efektif digunakan jika datanya sudah hampir berurutan, Insertion Sort lebih baik diterapkan pada data dalam jumlah sedikit, sedangkan Selection Sort memiliki kelebihan karena jumlah pertukaran datanya paling sedikit.

Adanya program berbasis antarmuka grafis ini juga memudahkan untuk menjelaskan cara kerja Insertion Sort kepada orang lain, karena setiap langkah proses ditampilkan secara nyata. Pengetahuan mengenai algoritma pengurutan ini sangat bermanfaat untuk membuat berbagai jenis aplikasi, misalnya untuk menyusun daftar nilai mahasiswa, daftar nama, maupun catatan transaksi.

Selain itu, praktikum ini juga melatih keterampilan saya dalam menemukan dan memperbaiki kesalahan program, mengingat pengaturan penunjuk data yang kurang tepat sering kali menjadi penyebab utama program tidak berjalan dengan semestinya.

BAB II PEMBAHASAN

2.1 Pengertian Algoritma Sorting

Pengurutan atau *Sorting* adalah kegiatan menyusun sekumpulan data menjadi urutan yang teratur, baik dimulai dari nilai paling kecil ke yang terbesar (menaik/ascending) maupun sebaliknya dari nilai terbesar ke yang terkecil (menurun/descending). Ada berbagai jenis metode pengurutan, dan tiga di antaranya yang dipelajari adalah Bubble Sort, Insertion Sort, serta Selection Sort.

Bubble Sort menjalankan prosesnya dengan membandingkan dua elemen yang letaknya berdekatan, lalu menukar posisinya jika susunannya belum sesuai. Langkah ini dilakukan berulang kali sebanyak $n-1$ putaran hingga seluruh data tersusun rapi. Sementara itu, cara kerja Insertion Sort mirip dengan kebiasaan kita saat menyusun kartu di tangan, yaitu mengambil satu bagian data dan menempatkannya pada posisi yang benar di antara kumpulan data yang sudah berurutan.

Berbeda dengan keduanya, Selection Sort bekerja dengan cara mencari elemen yang bernilai paling kecil dari bagian data yang belum tersusun, kemudian memindahkannya ke posisi paling awal. Ketiga algoritma ini memiliki tingkat kerumitan waktu sebesar $O(n^2)$ untuk kondisi terberat, sehingga penggunaannya paling tepat diterapkan pada data dengan jumlah yang tidak terlalu banyak.

2.2 Implementasi Pada Bahasa Java

Berikut ini adalah uraian penjelasan untuk setiap kode program pengurutan yang telah dipelajari dan dikerjakan selama praktikum.

2.2.1 BubbleSort_2511533004

```
1 package pekan7_2511533004;
2
3 public class BubbleSort_2511533004 {
4     public static void bubbleSort_3004(int[] arr) {
5         int n_3004 = arr.length;
6         for (int i_3004 = 0; i_3004 < n_3004; i_3004++) {
7             for (int j_3004 = 0; j_3004 < n_3004 - i_3004 - 1; j_3004++) {
8                 if (arr[j_3004] > arr[j_3004 + 1]) {
9                     int temp_3004 = arr[j_3004];
10                    arr[j_3004] = arr[j_3004 + 1];
11                    arr[j_3004 + 1] = temp_3004;
12                }
13                // System.out.println("data:" + arr[j] + " " + arr[j+1]);
14            }
15        }
16    }
17
18    public static void main(String[] args) {
19        int arr_3004[] = { 23, 78, 45, 8, 32, 56, 1 };
20        int n_3004 = arr_3004.length;
21        System.out.print("array yang belum terurut:");
22        for (int i = 0; i < n_3004; i++) {
23            System.out.print(arr_3004[i] + " ");
24        }
25        System.out.println("");
26        bubbleSort_3004(arr_3004);
27        System.out.print("array yang terurut menggunakan BubleSort:");
28        for (int i = 0; i < n_3004; i++) {
29            System.out.print(arr_3004[i] + " ");
30        }
31        System.out.println("");
32    }
33 }
```

Program ini menerapkan algoritma Bubble Sort untuk menyusun urutan data yang berupa kumpulan bilangan bulat. Fungsi bubbleSort akan menerima masukan berupa larik data bernama arr, lalu mengaturnya dari nilai terkecil ke terbesar. Cara kerja algoritma ini menggunakan dua pengulangan yang saling bersambung. Pengulangan

utama (`i_3004`) berfungsi mengatur berapa kali proses pengurutan dilakukan, sesuai dengan banyaknya data dalam larik. Sedangkan pengulangan kedua (`j_3004`) bertugas membandingkan tiap elemen yang berdekatan.

Elemen `arr[j_3004]` akan dibandingkan dengan `arr[j_3004 + 1]`. Jika nilai elemen di sebelah kiri lebih besar daripada elemen di sebelah kanan, maka posisi keduanya akan ditukar dengan bantuan variabel penyimpan sementara bernama `temp_3004`. Setelah satu kali proses pengulangan selesai, elemen dengan nilai terbesar akan otomatis berada di urutan paling kanan.

Di dalam fungsi utama `main`, dibuat sebuah larik data yang berisi nilai {23, 78, 45, 8, 32, 56, 1}. Data tersebut ditampilkan terlebih dahulu sebelum diurutkan, kemudian fungsi `bubbleSort` dijalankan, dan akhirnya susunan data diperlihatkan kembali setelah proses selesai. Hasil yang didapatkan menunjukkan bahwa larik yang tadinya [23, 78, 45, 8, 32, 56, 1] berubah menjadi urutan yang rapi, yaitu [1, 8, 23, 32, 45, 56, 78].

```
<terminated> BubleSort_2511533005 [Java Application] C:\Users\Lenovo\p2\po
array yang belum terurut:23 78 45 8 32 56 1
array yang terurut menggunakan BubleSort:1 8 23 32 45 56 78
```

2.2.2 InsertionSort_2511533004

```
1 package pekan7_2511533004;
2
3 public class InsertionSort_2511533004 {
4     public static void insertionSort_3004(int[] arr) {
5         int n_3004 = arr.length;
6         for (int i_3004 = 1; i_3004 < n_3004; i_3004++) {
7             int key_3004 = arr[i_3004];
8             int j_3004 = i_3004 - 1;
9             while (j_3004 >= 0 && arr[j_3004] > key_3004) {
10                 arr[j_3004 + 1] = arr[j_3004];
11                 j_3004--;
12             }
13             arr[j_3004 + 1] = key_3004;
14         }
15     }
16
17     public static void main(String[] args) {
18         int arr_3004[] = { 23, 78, 45, 8, 32, 56, 1 };
19         int n = arr_3004.length;
20         System.out.printf("array yang belum terurut:\n");
21         for (int i = 0; i < n; i++) {
22             System.out.print(arr_3004[i] + " ");
23         }
24         System.out.println("");
25         insertionSort_3004(arr_3004);
26         System.out.printf("array yang terurut:\n");
27         for (int i = 0; i < n; i++) {
28             System.out.print(arr_3004[i] + " ");
29         }
30         System.out.println("");
31     }
32 }
```

Program ini menerapkan algoritma Insertion Sort untuk mengurutkan kumpulan bilangan bulat dalam sebuah larik. Cara kerja fungsi `insertionSort` adalah dengan memisahkan data menjadi dua bagian, yaitu sisi kiri yang sudah tersusun rapi, dan sisi kanan yang belum diurutkan. Proses pengulangan dimulai dari indeks ke-1 sampai ke elemen paling akhir. Untuk setiap nilai `i_3004`, data yang akan diproses disimpan dalam variabel `key_3004` yang diambil dari posisi `arr[i_3004]`, kemudian variabel `j_3004` diatur bernilai `i_3004 - 1`. Selama nilai `j_3004` masih lebih dari atau sama dengan nol,

serta nilai `arr[j_3004]` lebih besar dibandingkan `key_3004`, maka posisi `arr[j_3004 + 1]` akan digeser ke arah kanan dan nilai `j_3004` dikurangi satu angka.

Setelah tahapan pengulangan selesai, nilai `key_3004` diletakkan pada posisi `arr[j_3004 + 1]`. Langkah ini dilakukan berulang kali hingga seluruh elemen data tersusun dengan rapi. Pada fungsi utama `main`, digunakan kumpulan data yang sama persis seperti pada algoritma Bubble Sort. Data tersebut ditampilkan terlebih dahulu sebelum diproses, lalu diperlihatkan kembali setelah proses pengurutan selesai. Hasil yang diperoleh pun serupa, yaitu susunan data yang teratur dari nilai terkecil ke terbesar.

```
<terminated> InsertionSort_2511533
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.2.3 SelectionSort_2511533004

```
1 package pekan7_2511533004;
2
3 public class SelectionSort_2511533004 {
4     public static void selectionSort_3004(int[] arr) {
5         int n_3004 = arr.length;
6         for (int i_3004 = 0; i_3004 < n_3004; i_3004++) {
7             int minIndex_3004 = i_3004;
8             for (int j_3004 = i_3004 + 1; j_3004 < n_3004; j_3004++) {
9                 if (arr[j_3004] < arr[minIndex_3004]) {
10                     minIndex_3004 = j_3004;
11                 }
12             }
13             int temp = arr[i_3004];
14             arr[i_3004] = arr[minIndex_3004];
15             arr[minIndex_3004] = temp;
16         }
17     }
18
19     public static void main(String[] args) {
20         int arr_3004[] = { 23, 78, 45, 8, 32, 56, 1 };
21         int n_3004 = arr_3004.length;
22         System.out.printf("array yang belum terurut:\n");
23         for (int i = 0; i < n_3004; i++) {
24             System.out.print(arr_3004[i] + " ");
25         }
26         System.out.println("");
27         selectionSort_3004(arr_3004);
28         System.out.printf("array yang terurut:\n");
29         for (int i = 0; i < n_3004; i++) {
30             System.out.print(arr_3004[i] + " ");
31         }
32         System.out.println("");
33     }
34 }
35
```

Program ini menerapkan algoritma Selection Sort untuk proses pengurutan data. Fungsi `selectionSort` bekerja dengan cara mencari nilai paling kecil dari bagian data yang belum tersusun, lalu memindahkannya ke posisi yang seharusnya. Pengulangan utama (`i_3004`) berjalan mulai dari elemen paling awal hingga akhir larik. Untuk setiap nilai `i_3004`, variabel `minIndex_3004` ditetapkan terlebih dahulu sama dengan `i_3004`. Selanjutnya, pengulangan kedua (`j_3004`) dimulai dari posisi `i_3004 + 1` sampai ke elemen terakhir. Jika ditemukan data pada `arr[j_3004]` yang nilainya lebih kecil dibandingkan `arr[minIndex_3004]`, maka nilai `minIndex_3004` akan diperbarui. Setelah proses pengulangan kedua selesai, elemen pada indeks `i_3004` ditukar posisinya dengan

elemen di indeks minIndex_3004 menggunakan variabel penyimpanan sementara bernama temp_3004.

Melalui langkah ini, elemen dengan nilai terkecil akan diletakkan pada urutan paling depan, diikuti oleh elemen terkecil kedua pada posisi berikutnya, dan seterusnya hingga seluruh data tersusun rapi. Pada fungsi utama main, digunakan kumpulan bilangan yang sama seperti pada algoritma sebelumnya, sehingga hasil yang didapatkan pun serupa, yaitu susunan data yang sudah terurut dengan baik.

```
<terminated> SelectionSort_2511533004  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut:  
1 8 23 32 45 56 78
```

2.2.4 InsertionSortGUI_2511533004

```
1 package pekan7_2511533004;  
2  
3 import java.awt.BorderLayout;  
4 import java.awt.FlowLayout;  
5 import java.awt.Font;  
6 import java.awt.Color;  
7 import java.awt.Dimension;  
8 import javax.swing.JButton;  
9 import javax.swing.JFrame;  
10 import javax.swing.JLabel;  
11 import javax.swing.JPanel;  
12 import javax.swing.JTextArea;  
13 import javax.swing.JTextField;  
14 import javax.swing.JScrollPane;  
15 import javax.swing.border.EmptyBorder;  
16 import javax.swing.JOptionPane;  
17 import javax.swing.BorderFactory;  
18 import javax.swing.SwingConstants;  
19 import javax.swing.SwingUtilities;  
20  
21 public class GUIInsertionSort_2511533004 extends JFrame {  
22  
23     private static final long serialVersionUID = 1L;  
24     private int[] array_3004;  
25     private JLabel[] labelArray_3004;  
26     private JButton stepButton_3004, resetButton_3004, setButton_3004;  
27     private JTextField inputField_3004;  
28     private JPanel panelArray_3004, controlPanel, inputPanel;  
29     private JTextArea stepArea_3004;  
30     private int i = 1, j_3004;  
31     private boolean sorting_3004 = false;  
32     private int stepCount_3004 = 1;  
33  
34     public GUIInsertionSort_2511533004() {  
35         setTitle("Insertion Sort Langkah per Langkah");  
36         setSize(800, 500);  
37  
38         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
39         setLocationRelativeTo(null);  
40         setLayout(new BorderLayout());  
41  
42         inputPanel = new JPanel(new FlowLayout());  
43         inputField_3004 = new JTextField(20);  
44         setButton_3004 = new JButton("Set Data");  
45         inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));  
46         inputPanel.add(inputField_3004);  
47         inputPanel.add(setButton_3004);  
48  
49         panelArray_3004 = new JPanel();  
50         panelArray_3004.setLayout(new FlowLayout());  
51  
52         controlPanel = new JPanel();  
53         stepButton_3004 = new JButton("Langkah Selanjutnya");  
54         resetButton_3004 = new JButton("Reset");  
55         stepButton_3004.setEnabled(false);  
56         controlPanel.add(stepButton_3004);  
57         controlPanel.add(resetButton_3004);  
58  
59         stepArea_3004 = new JTextArea(8, 60);  
60         stepArea_3004.setEditable(false);  
61         stepArea_3004.setFont(new Font("Monospaced", Font.PLAIN, 14));  
62         JScrollPane scrollPane_3004 = new JScrollPane(stepArea_3004);  
63  
64         add(inputPanel, BorderLayout.NORTH);  
65         add(panelArray_3004, BorderLayout.CENTER);  
66         add(controlPanel, BorderLayout.SOUTH);  
67         add(scrollPane_3004, BorderLayout.EAST);  
68  
69         setButton_3004.addActionListener(e -> setArrayFromInput_3004());  
70         stepButton_3004.addActionListener(e -> performStep_3004());  
71         resetButton_3004.addActionListener(e -> reset_3004());  
72     }  
73 }
```



```

73 private void setArrayFromInput_3004() {
74     String text = inputField_3004.getText().trim();
75     if (text.isEmpty()) return;
76     String[] parts = text.split(",");
77     array_3004 = new int[parts.length];
78
79     try {
80         for (int k = 0; k < parts.length; k++) {
81             array_3004[k] = Integer.parseInt(parts[k].trim());
82         }
83     } catch (NumberFormatException e) {
84         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
85         return;
86     }
87
88     i = 1;
89     stepCount_3004 = 1;
90     sorting_3004 = true;
91     stepButton_3004.setEnabled(true);
92     stepArea_3004.setText("");
93     panelArray_3004.removeAll();
94
95     labelArray_3004 = new JLabel[array_3004.length];
96     for (int k = 0; k < array_3004.length; k++) {
97         labelArray_3004[k] = new JLabel(String.valueOf(array_3004[k]));
98         labelArray_3004[k].setFont(new Font("Arial", Font.BOLD, 24));
99         labelArray_3004[k].setBorder(BorderFactory.createLineBorder(color.BLACK));
100        labelArray_3004[k].setPreferredSize(new Dimension(50, 50));
101        labelArray_3004[k].setHorizontalAlignment(SwingConstants.CENTER);
102        panelArray_3004.add(labelArray_3004[k]);
103    }
104    panelArray_3004.revalidate();
105    panelArray_3004.repaint();
106 }
107
108 private void performStep_3004() {
109     if (i < array_3004.length && sorting_3004) {
110         int key = array_3004[i];
111         j_3004 = i - 1;
112
113         StringBuilder stepLog = new StringBuilder();
114         stepLog.append("Langkah ").append(stepCount_3004)
115             .append(": Memasukkan ").append(key).append("\n");
116
117         while (j_3004 >= 0 && array_3004[j_3004] > key) {
118             array_3004[j_3004 + 1] = array_3004[j_3004];
119             j_3004--;
120         }
121         array_3004[j_3004 + 1] = key;
122
123         updateLabels_3004();
124         stepLog.append("Hasil: ").append(arrayToString_3004(array_3004)).append("\n\n");
125         stepArea_3004.append(stepLog.toString());
126
127         i++;
128         stepCount_3004++;
129
130         if (i == array_3004.length) {
131             sorting_3004 = false;
132             stepButton_3004.setEnabled(false);
133             JOptionPane.showMessageDialog(this, "Sorting selesai!");
134         }
135     }
136 }
137
138 private void updateLabels_3004() {
139     for (int k = 0; k < array_3004.length; k++) {
140         labelArray_3004[k].setText(String.valueOf(array_3004[k]));
141     }
142 }
143
144 private void reset_3004() {
145     inputField_3004.setText("");
146     panelArray_3004.removeAll();
147     panelArray_3004.revalidate();
148     panelArray_3004.repaint();
149     stepArea_3004.setText("");
150     stepButton_3004.setEnabled(false);
151     sorting_3004 = false;
152     i = 1;
153     stepCount_3004 = 1;
154 }
155
156 private String arrayToString_3004(int[] arr) {
157     StringBuilder sb = new StringBuilder();
158     for (int k = 0; k < arr.length; k++) {
159         sb.append(arr[k]);
160         if (k < arr.length - 1) sb.append(", ");
161     }
162     return sb.toString();
163 }
164
165 public static void main(String[] args) {
166     SwingUtilities.invokeLater(() -> {
167         GUIInsertionsort_2511533004 gui = new GUIInsertionsort_2511533004();
168         gui.setVisible(true);
169     });
170 }
171 }

```

Program ini merupakan antarmuka grafis atau **GUI** yang berfungsi menampilkan tahapan proses pengurutan menggunakan algoritma Insertion Sort secara bertahap. Tampilan ini dikembangkan dengan memanfaatkan pustaka **Java Swing**. Di dalamnya terdapat berbagai elemen pendukung, yaitu: kolom input **JTextField** untuk memasukkan deretan angka yang dipisahkan dengan tanda koma, tombol “*Tetapkan Data*” untuk mengolah masukan yang diberikan, tombol “*Langkah Berikutnya*” untuk menjalankan satu tahapan pengurutan setiap kali ditekan, tombol “*Atur Ulang*” untuk

mengembalikan kondisi program seperti semula, area tampilan JPanel untuk menampilkan susunan data dalam bentuk kotak-kotak visual, serta kolom teks JTextArea yang berfungsi mencatat dan menampilkan keterangan setiap langkah yang berlangsung.

Ketika pengguna menekan tombol “*Tetapkan Data*”, masukan yang dimasukkan akan diubah menjadi bentuk larik bilangan bulat, lalu langsung ditampilkan dalam bentuk kotak-kotak berisi nilai masing-masing elemen. Sementara itu, tombol “*Langkah Berikutnya*” akan

tetap berfungsi. Setiap kali tombol tersebut diklik, sistem akan melaksanakan satu tahapan pengurutan sesuai aturan Insertion Sort, yaitu menempatkan satu elemen data ke urutan yang seharusnya di bagian yang sudah tersusun. Keterangan setiap proses yang terjadi akan tercatat dan ditampilkan pada kolom teks JTextArea, sedangkan tampilan kotak-kotak angka di area panel akan diperbarui secara otomatis mengikuti perubahan posisi elemen. Langkah ini akan terus berjalan hingga seluruh data tersusun rapi. Setelah proses selesai, tombol untuk melanjutkan langkah akan dinonaktifkan, dan muncul pemberitahuan bertuliskan “**Proses pengurutan selesai!**”.

Dengan adanya tampilan visual dan penjelasan setiap tahapnya, program ini menjadi sarana yang sangat efektif untuk memudahkan pengguna memahami cara kerja serta alur logika algoritma Insertion Sort secara lebih jelas dan mudah dicerna.

TUGAS PRATIUM

Mahasiswa_2511533004

```
1 package pekan7_2511533004;
2
3 public class Mahasiswa_2511533004 {
4     private String nama_3004;
5     private String nim_3004;
6     private String prodi_3004;
7
8     // constructor
9     public Mahasiswa_2511533004(String nama_3004, String nim_3004, String prodi_3004) {
10         this.nama_3004 = nama_3004;
11         this.nim_3004 = nim_3004;
12         this.prodi_3004 = prodi_3004;
13     }
14
15     // getter setter
16     public String getNama() {
17         return nama_3004;
18     }
19
20     public void setNama(String nama_3004) {
21         this.nama_3004 = nama_3004;
22     }
23
24     public String getNim() {
25         return nim_3004;
26     }
27
28     public void setNim(String nim_3004) {
29         this.nim_3004 = nim_3004;
30     }
31
32     public String getProdi() {
33         return prodi_3004;
34     }
35
36     public void setProdi(String prodi) {
37         this.prodi_3004 = prodi;
38     }
39
40     @Override
41     public String toString() {
42         return nama_3004 + " - " + nim_3004 + " - " + prodi_3004;
43     }
44 }
```

Sorting_2511533004

```
1 package pekan7_2511533004;
2
3 import java.awt.BorderLayout;
4 import java.awt.Dimension;
5 import java.awt.EventQueue;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8 import java.awt.GridLayout;
9 import java.awt.event.ActionEvent;
10 import java.awt.event.ActionListener;
11 import java.util.ArrayList;
12
13 import javax.swing.BorderFactory;
14 import javax.swing.JButton;
15 import javax.swing.JComboBox;
16 import javax.swing.JFrame;
17 import javax.swing.JPanel;
18 import javax.swing.JScrollPane;
19 import javax.swing.JTable;
20 import javax.swing.JTextArea;
21 import javax.swing.border.EmptyBorder;
22 import javax.swing.table.DefaultTableModel;
23 import javax.swing.JLabel;
24 import javax.swing.JOptionPane;
25 import javax.swing.JTextField;
26
27 public class SortingGUI_2511533004 extends JFrame {
28
29     private static final long serialVersionUID = 1L;
30     private JPanel contentPane;
31
32     // 1. DEKLARASI KOMPONEN GUI
33     private JTextField txtNama_3004, txtNim_3004, txtProdi_3004;
34     private JButton btnTambah_3004, btnHapus_3004, btnSort_3004;
35     private JComboBox<String> cmbAlgoritma_3004;
36     private JTable tabelData_3004;
```

```

37 private DefaultTableModel tableModel_3004;
38 private JTextArea txtAreaLog_3004;
39
40 // Jemput menyimpan database Mahasiswa sementara di memori
41 private ArrayList<Mahasiswa_2511533004> listMahasiswa;
42
43 /**
44  * Create the frame.
45  */
46 public SortingGUI_2511533004() {
47     // 2. PENGATURAN FRAME UTAMA
48     setTitle("Pengurutan Nama Mahasiswa - Pekan 7");
49     setSize(850, 650);
50     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
51     setLocationRelativeTo(null); // Memuat window muncul di tengah layar.
52
53     listMahasiswa = new ArrayList<>();
54
55     // Inisialisasi contentPane
56     contentPane = new JPanel();
57     contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
58     setContentPane(contentPane);
59     contentPane.setLayout(new BorderLayout(15, 15));
60
61     // 3. MEMBUAT PANEL INPUT (Bagian Atas)
62     JPanel panelInput = new JPanel(new GridLayout(4, 2, 10, 10));
63     panelInput.setBorder(BorderFactory.createTitledBorder("Form Input Mahasiswa"));
64
65     panelInput.add(new JLabel(" NIM Mahasiswa:"));
66     txtNim_3004 = new JTextField();
67     panelInput.add(txtNim_3004);
68
69     panelInput.add(new JLabel(" Nama Mahasiswa:"));
70     txtNama_3004 = new JTextField();
71     panelInput.add(txtNama_3004);
72
73     panelInput.add(new JLabel(" Program Studi:"));
74     txtProdi_3004 = new JTextField();
75     panelInput.add(txtProdi_3004);
76
77     btnTambah_3004 = new JButton("Tambah Data");
78     btnHapus_3004 = new JButton("Kosongkan Semua Data");
79     panelInput.add(btnTambah_3004);
80     panelInput.add(btnHapus_3004);
81     contentPane.add(panelInput, BorderLayout.NORTH);
82
83     // 4. MEMBUAT PANEL KONTROL & TABEL (Bagian Tengah)
84     JPanel panelTengah = new JPanel(new BorderLayout(10, 10));
85
86     // Sub-panel untuk pilihan dropdown dan tombol urutkan
87     JPanel panelSaring = new JPanel(new FlowLayout(FlowLayout.LEFT));
88     panelSaring.add(new JLabel("Pilih Algoritma Sorting: "));
89     String[] pilihanAlgoritma = {"Insertion Sort", "Selection Sort", "Bubble Sort"};
90     cmbAlgoritma_3004 = new JComboBox<>(pilihanAlgoritma);
91     panelSaring.add(cmbAlgoritma_3004);
92
93     btnSort_3004 = new JButton("Mulai Sorting");
94     panelSaring.add(btnSort_3004);
95     panelTengah.add(panelSaring, BorderLayout.NORTH);
96
97     // Membuat JTable untuk menampilkan data
98     String[] kolom = {"NIM", "Nama Mahasiswa", "Program Studi"};
99     tableModel_3004 = new DefaultTableModel(kolom, 0);
100     tabelData_3004 = new JTable(tableModel_3004);
101     JScrollPane scrollTabel = new JScrollPane(tabelData_3004);
102     scrollTabel.setPreferredSize(new Dimension(800, 150));
103     panelTengah.add(scrollTabel, BorderLayout.CENTER);
104     contentPane.add(panelTengah, BorderLayout.CENTER);
105
106     // 5. MEMBUAT PANEL LOG VISUALISASI (Bagian Bawah)
107     JPanel panelBawah = new JPanel(new BorderLayout());
108     panelBawah.setBorder(BorderFactory.createTitledBorder("Visualisasi Proses Langkah demi Langkah"));

```

```

109         txtAreaLog_3004 = new JTextArea();
110         txtAreaLog_3004.setEditable(false); // Log tidak bisa diedit manual
111         txtAreaLog_3004.setFont(new Font("Monospaced", Font.PLAIN, 13)); // Font agar tampilan rapi
112         JScrollPane scrollLog = new JScrollPane(txtAreaLog_3004);
113         scrollLog.setPreferredSize(new Dimension(800, 220));
114         panelBawah.add(scrollLog, BorderLayout.CENTER);
115         contentPane.add(panelBawah, BorderLayout.SOUTH);
116
117         // 7. LOGIKA AKSI TOMBOL
118
119         // Aksi Tombol Tambah
120         btnTambah_3004.addActionListener(new ActionListener() {
121             public void actionPerformed(ActionEvent e) {
122                 tambahDataKeList();
123             }
124         });
125
126         // Aksi Tombol Hapus
127         btnHapus_3004.addActionListener(new ActionListener() {
128             @Override
129             public void actionPerformed(ActionEvent e) {
130                 listMahasiswa.clear();
131                 refreshTabel();
132                 txtAreaLog_3004.setText("");
133             }
134         });
135
136         // Aksi Tombol Jalankan Sorting
137         btnSort_3004.addActionListener(new ActionListener() {
138             @Override
139             public void actionPerformed(ActionEvent e) {
140                 eksekusiSorting();
141             }
142         });
143     });
144 }
145
146 // METHOD AMBIL INPUT DAN MASUKKAN KE ARRAYLIST
147 private void tambahDataKeList() {
148     String nim = txtNim_3004.getText().trim();
149     String nama = txtNama_3004.getText().trim();
150     String prodi = txtProdi_3004.getText().trim();
151
152     // Validasi jika ada kolom yang kosong
153     if (nim.isEmpty() || nama.isEmpty() || prodi.isEmpty()) {
154         JOptionPane.showMessageDialog(this, "Isi semua data terlebih dahulu!", "Peringatan", JOptionPane.WARNING_MESSAGE);
155         return;
156     }
157
158     // Buat objek mahasiswa baru
159     Mahasiswa_2511533004 mhs = new Mahasiswa_2511533004(nama, nim, prodi);
160     listMahasiswa.add(mhs);
161
162     // Refresh tampilan tabel
163     refreshTabel();
164
165     // Kosongkan kolom input
166     txtNim_3004.setText("");
167     txtNama_3004.setText("");
168     txtProdi_3004.setText("");
169     txtNim_3004.requestFocus();
170 }
171
172 // METHOD MEMPERBARUI ISI TABEL
173 private void refreshTabel() {
174     tableModel_3004.setRowCount(0); // Hapus semua data lama
175     for (Mahasiswa_2511533004 mhs : listMahasiswa) {
176         tableModel_3004.addRow(new Object[]{mhs.getNim(), mhs.getNama(), mhs.getProdi()});
177     }
178 }
179
180 // METHOD MEMILIH DAN MENJALANKAN ALGORITMA
181 private void eksekusiSorting() {
182     if (listMahasiswa.size() < 2) {
183         JOptionPane.showMessageDialog(this, "Masukkan minimal 2 data mahasiswa untuk diurutkan!", "Info", JOptionPane.INFORMATION_MESSAGE);
184         return;
185     }
186
187     // Salin data agar data asli tidak berubah sebelum dicoba metode lain
188     ArrayList<Mahasiswa_2511533004> dataUrut = new ArrayList<>(listMahasiswa);
189     txtAreaLog_3004.setText(""); // Bersihkan log sebelumnya
190
191     String algoritmaTerpilih = (String) cmbAlgoritma_3004.getSelectedItem();
192
193     // Jalankan algoritma sesuai pilihan
194     if (algoritmaTerpilih.equals("Insertion Sort")) {
195         jalankanInsertionSort(dataUrut);
196     } else if (algoritmaTerpilih.equals("Selection Sort")) {
197         jalankanSelectionSort(dataUrut);
198     } else if (algoritmaTerpilih.equals("Bubble Sort")) {
199         jalankanBubbleSort(dataUrut);
200     }
201
202     // Simpan hasil pengurutan dan tampilkan
203     listMahasiswa = dataUrut;
204     refreshTabel();
205 }
206
207 // =====
208 // IMPLEMENTASI SETIAP ALGORITMA SORTING
209 // =====
210
211 private void jalankanInsertionSort(ArrayList<Mahasiswa_2511533004> arr) {
212     txtAreaLog_3004.append("=== INSERTION SORT ===\n");
213     txtAreaLog_3004.append("Data Awal : " + arr.toString() + "\n\n");
214
215     int n = arr.size();
216     for (int i = 1; i < n; i++) {

```

```

218         int j = i - 1;
219
220         while (j >= 0 && arr.get(j).getNama().compareToIgnoreCase(key.getNama()) > 0) {
221             arr.set(j + 1, arr.get(j));
222             j = j - 1;
223         }
224         arr.set(j + 1, key);
225
226         txtAreaLog_3004.append("Langkah " + i + " : " + arr.toString() + "\n");
227     }
228     txtAreaLog_3004.append("\n☑ Pengurutan selesai!");
229 }
230
231 private void jalankanSelectionSort(ArrayList<Mahasiswa_2511533004> arr) {
232     txtAreaLog_3004.append("=== SELECTION SORT ===\n");
233     txtAreaLog_3004.append("Data Awal : " + arr.toString() + "\n\n");
234
235     int n = arr.size();
236     for (int i = 0; i < n - 1; i++) {
237         int indeksMinimal = i;
238         for (int j = i + 1; j < n; j++) {
239             if (arr.get(j).getNama().compareToIgnoreCase(arr.get(indeksMinimal).getNama()) < 0) {
240                 indeksMinimal = j;
241             }
242         }
243         // Tukar posisi data
244         Mahasiswa_2511533004 temp = arr.get(indeksMinimal);
245         arr.set(indeksMinimal, arr.get(i));
246         arr.set(i, temp);
247
248         txtAreaLog_3004.append("Putaran " + (i + 1) + " : " + arr.toString() + "\n");
249     }
250     txtAreaLog_3004.append("\n☑ Pengurutan selesai!");
251 }
252
253 private void jalankanBubbleSort(ArrayList<Mahasiswa_2511533004> arr) {
254     txtAreaLog_3004.append("=== BUBBLE SORT ===\n");
255     txtAreaLog_3004.append("Data Awal : " + arr.toString() + "\n\n");
256
257     int n = arr.size();
258     for (int i = 0; i < n - 1; i++) {
259         for (int j = 0; j < n - i - 1; j++) {
260             if (arr.get(j).getNama().compareToIgnoreCase(arr.get(j + 1).getNama()) > 0) {
261                 // Tukar posisi data
262                 Mahasiswa_2511533004 temp = arr.get(j);
263                 arr.set(j, arr.get(j + 1));
264                 arr.set(j + 1, temp);
265             }
266         }
267         txtAreaLog_3004.append("Putaran " + (i + 1) + " : " + arr.toString() + "\n");
268     }
269     txtAreaLog_3004.append("\n☑ Pengurutan selesai!");
270 }
271
272 // METHOD UTAMA
273 public static void main(String[] args) {
274     EventQueue.invokeLater(new Runnable() {
275         public void run() {
276             try {
277                 SortingGUI_2511533004 frame = new SortingGUI_2511533004();
278                 frame.setVisible(true);
279             } catch (Exception e) {
280                 e.printStackTrace();
281             }
282         }
283     });
284 }

```

Output

Pengurutan Nama Mahasiswa - Pekan 7

Form Input Mahasiswa

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Tambah Data

Kosongkan Semua Data

Pilih Algoritma Sorting:

Insertion Sort

Mulai Sorting

NIM	Nama Mahasiswa	Program Studi
3037	dara	pertanian
3027	hanafi	pertanian
1199	nindhy	hukum
3004	silva	informatika

Visualisasi Proses Langkah demi Langkah

=== INSERTION SORT ===
Data Awal : [silva - 3004 - informatika, dara - 3037 - pertanian, nindhy - 1199 - hukum, hanafi - 3027 - pertanian]

Langkah 1 : [dara - 3037 - pertanian, silva - 3004 - informatika, nindhy - 1199 - hukum, hanafi - 3027 - pertanian]
Langkah 2 : [dara - 3037 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika, hanafi - 3027 - pertanian]
Langkah 3 : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]

✓ Pengurutan selesai!

Pengurutan Nama Mahasiswa - Pekan 7

Form Input Mahasiswa

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Tambah Data

Kosongkan Semua Data

Pilih Algoritma Sorting:

Selection Sort

Mulai Sorting

NIM	Nama Mahasiswa	Program Studi
3037	dara	pertanian
3027	hanafi	pertanian
1199	nindhy	hukum
3004	silva	informatika

Visualisasi Proses Langkah demi Langkah

=== SELECTION SORT ===
Data Awal : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]

Putaran 1 : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]
Putaran 2 : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]
Putaran 3 : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]

✓ Pengurutan selesai!

Pengurutan Nama Mahasiswa - Pekan 7

Form Input Mahasiswa

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Tambah Data

Kosongkan Semua Data

Pilih Algoritma Sorting:

Bubble Sort

Mulai Sorting

NIM	Nama Mahasiswa	Program Studi
3037	dara	pertanian
3027	hanafi	pertanian
1199	nindhy	hukum
3004	silva	informatika

Visualisasi Proses Langkah demi Langkah

=== BUBBLE SORT ===

Data Awal : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]

Putaran 1 : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]

Putaran 2 : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]

Putaran 3 : [dara - 3037 - pertanian, hanafi - 3027 - pertanian, nindhy - 1199 - hukum, silva - 3004 - informatika]

✓ Pengurutan selesai!